МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»
Институт перспективной инженерии
Межинститутская базовая кафедра
Институт информационных технологий и кибербезопасности

Отчёт к лабораторной работе №6

Тема: **«Тестирование по типу приложений»**

Дисциплина: **«Тестирование и отладка программного обеспечения»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки:
09.03.04 «Программная
инженерия»

Астраков Борис Александрович

_____

Проверил:

Ассистент МИБК

Потапов Иван Романович

_____

Ставрополь 2026

Цель работы: для разработанного приложения продемонстрировать применение методологии тестирования: тестирование мобильного приложения.

Ход работы:

Вариант проекта – мобильное приложение цифрового гардероба (Java + Kotlin).

Приложение разработано на основе архитектурного паттерна PCMEF (Presentation, Control, Mediator, Entity, Foundation), и состоит из сервера Spring Boot и мобильного приложения на Kotlin. Для серверной части применим методологию Unit-тестирования, а мобильное приложение будем тестировать вручную, через UI.

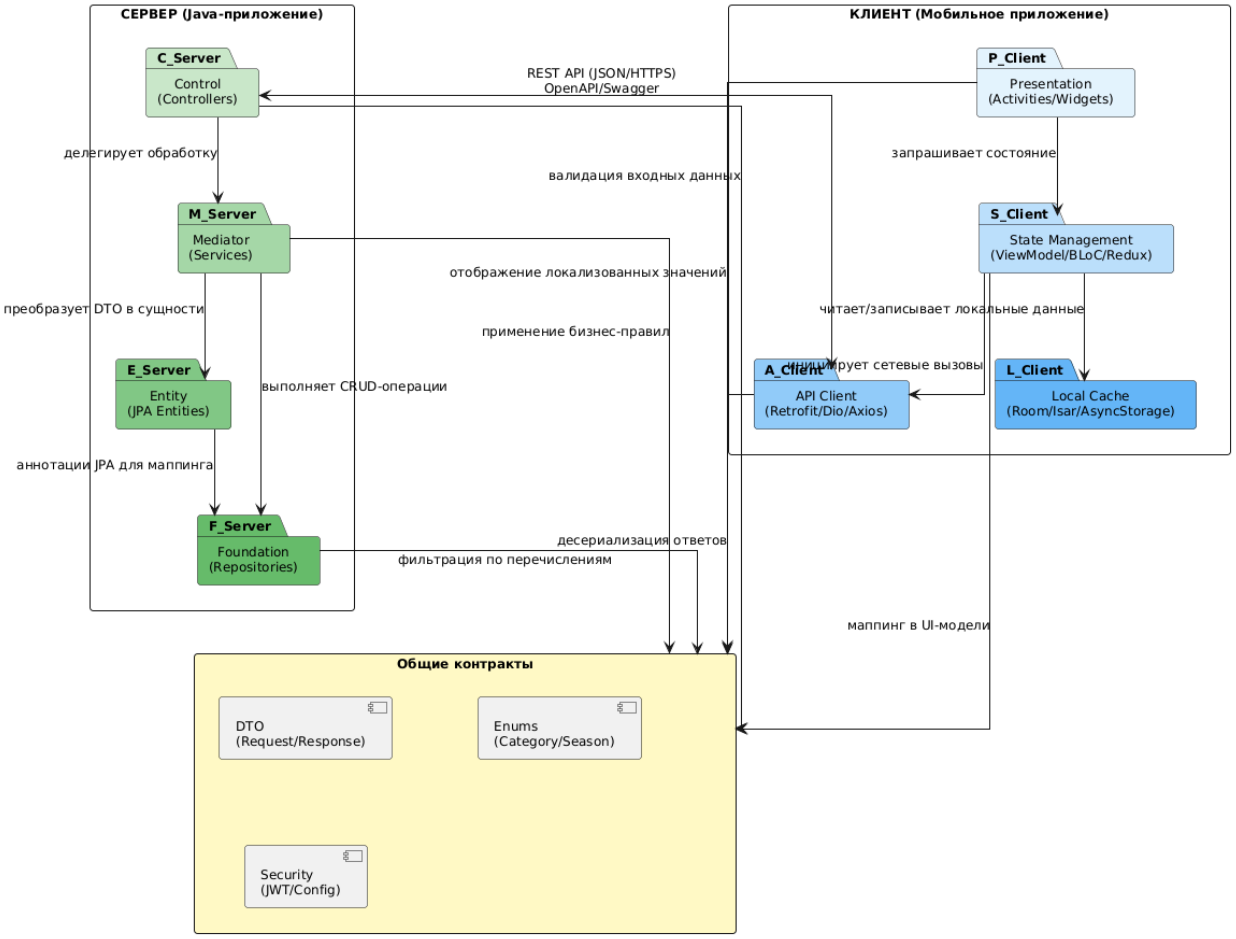


Рисунок 1. Диаграмма зависимостей проекта

Часть 1. Unit-тесты для сервера

Сервер приложения «Личный гардероб» — это REST-бэкенд на Spring Boot, который хранит пользователей, их вещи и составленные из вещей образы. Он обеспечивает регистрацию и вход с выдачей JWT-токена, проверку прав доступа и работу каждого пользователя только со своими данными. Вещи описываются названием, категорией и сезоном, а образы состоят из слотов, ссылающихся на вещи или остающихся пустыми. При сохранении данных сервис проверяет валидность полей, принадлежность вещи пользователю и соответствие категории вещи типу слота, а ошибки возвращает в виде понятных HTTP-статусов.

Требования к серверу:

1) Аутентификация и пользователи:

- регистрация по email/паролю с уникальным email; при успехе — выдача JWT-токена и профиля;

- вход по email/паролю с единой ошибкой при неверных данных;

- смена пароля с проверкой текущего;

- просмотр профиля (id, email, имя, аватар);

- все защищённые эндпоинты требуют аутентификации (JWT), иначе — 401.

2) Вещи:

- полный CRUD над вещами пользователя (список, просмотр, создание, изменение, удаление).

- вещь: название, описание, изображение, категория, сезон.

- обязательные поля валидируются (название, категория); некорректные данные → 400.

- пользователь работает только со своими вещами; чужие — недоступны.

3) Образы:

- полный CRUD над образами пользователя (список, просмотр, создание, изменение, удаление);

- образ состоит из слотов; слот привязан к типу категории и может быть пустым (шаблон);

- обязательные поля валидируются (название, непустой набор слотов); некорректные данные = 400;

- бизнес-правила: слот ссылается на существующую вещь текущего пользователя, и её категория совпадает с типом слота;

- пользователь работает только со своими образами; чужие — недоступны;

- отсутствующие сущности = ошибка «не найдено»; нарушение прав = отказ в доступе.

Составим тесты для слоёв Control, (controller), Entity (jpa) и Mediator (service) согласно вышеперечисленным требованиям.

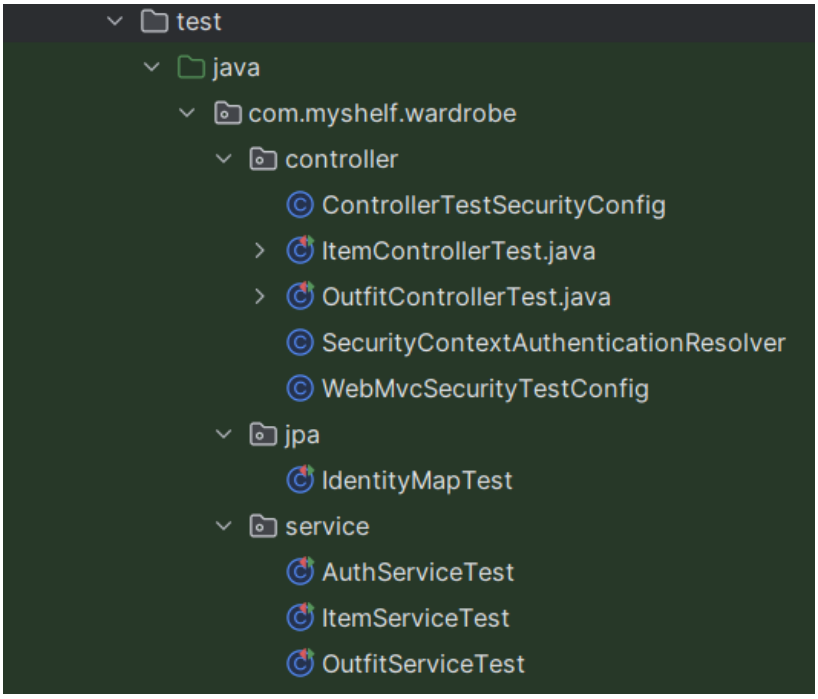


Рисунок 2. Структура тестов сервера Spring Boot

Примеры тестов для каждого слоя:

1) ItemControllerTest.java — создание вещи (успешный сценарий)

```
@Test
@DisplayName("POST /api/items – 201 Created")
void createItem_returnsCreated() throws Exception {
    ItemDTO requestDto = new ItemDTO(null, "New Item", "Description", null, Category.
TOP, Season.WINTER);
    ItemDTO createdDto = new ItemDTO(itemId, "New Item", "Description", null, Categor
y.TOP, Season.WINTER);
    when(itemService.createItem(eq(userId), any(ItemDTO.class))).thenReturn(createdDt
o);

    mockMvc.perform(post("/api/items")
                    .with(authenticatedUser())
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(requestDto)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value(itemId.toString()))
            .andExpect(jsonPath("$.name").value("New Item"));

    verify(itemService).createItem(eq(userId), any(ItemDTO.class));
}
```

2) OutfitControllerTest.java — создание образа со слотами

```
@Test
@DisplayName("POST /api/outfits – 201 Created")
void createOutfit_returnsCreated() throws Exception {
    OutfitSlotDTO slotRequest = new OutfitSlotDTO(null, itemId, Category.TOP);
    OutfitDTO requestDto = new OutfitDTO(null, "New Outfit", "Description", List.of(s
lotRequest));
    OutfitDTO createdDto = new OutfitDTO(outfitId, "New Outfit", "Description", List.
of(slotRequest));
    when(outfitService.createOutfit(eq(userId), any(OutfitDTO.class))).thenReturn(cre
atedDto);

    mockMvc.perform(post("/api/outfits")
                    .with(authenticatedUser())
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(objectMapper.writeValueAsString(requestDto)))
            .andExpect(status().isCreated())
            .andExpect(jsonPath("$.id").value(outfitId.toString()))
            .andExpect(jsonPath("$.name").value("New Outfit"));
```

```
    verify(outfitService).createOutfit(eq(userId), any(OutfitDTO.class));
}
```

3) IdentityMapTest.java — Identity Map Hibernate

```
@Test
@DisplayName("EntityManager: две загрузки по id — один объект (Identity Map)")
void entityManager_returnsSameManagedInstanceForSameId() {
    Item first = entityManager.find(Item.class, itemId);
    Item second = entityManager.find(Item.class, itemId);

    assertSame(first, second, "Identity Map: обе ссылки должны указывать на один mana
ged-объект");

    first.setName("Changed via first reference");
    entityManager.flush();

    assertThat(second.getName())
            .as("изменение через first видно в second — общий экземпляр в Persistence
Context")
            .isEqualTo("Changed via first reference");
}
```

4) OutfitServiceTest.java — создание образа с валидными слотами

```
@Test
@DisplayName("createOutfit — успешное создание образа с валидными слотами")
void createOutfit_success() {
    // Arrange
    OutfitSlotDTO slotDTO = new OutfitSlotDTO(null, itemId, Category.TOP);
    OutfitDTO outfitDTO = new OutfitDTO(null, "Summer Outfit", "Casual summer look",
List.of(slotDTO));

    when(userRepository.findById(userId)).thenReturn(Optional.of(testUser));
    when(itemRepository.findById(itemId)).thenReturn(Optional.of(testItem));
    when(outfitRepository.save(any(Outfit.class))).thenAnswer(invocation -> {
        Outfit saved = invocation.getArgument(0);
        saved.setId(outfitId);
        return saved;
    });

    // Act
    OutfitDTO result = outfitService.createOutfit(userId, outfitDTO);
```

```
    // Assert
    assertThat(result).isNotNull();
    assertThat(result.getName()).isEqualTo("Summer Outfit");
    assertThat(result.getDescription()).isEqualTo("Casual summer look");
    assertThat(result.getSlots()).hasSize(1);
    assertThat(result.getSlots().get(0).getSlotType()).isEqualTo(Category.TOP);
    assertThat(result.getSlots().get(0).getItemId()).isEqualTo(itemId);

    verify(userRepository).findById(userId);
    verify(itemRepository).findById(itemId);
    verify(outfitRepository).save(any(Outfit.class));
}
```

5) AuthServiceTest.java — успешная регистрация

```
@Test
@DisplayName("register — успешная регистрация нового пользователя")
void register_success() {
    // Arrange
    UserRegistrationDTO regDTO = new UserRegistrationDTO(testEmail, testPassword, tes
tDisplayName);
    String expectedToken = "jwt.token.here";

    when(userRepository.existsByEmail(testEmail)).thenReturn(false);
    when(passwordEncoder.encode(testPassword)).thenReturn("$2a$10$hashedPassword");
    when(userRepository.save(any(User.class))).thenAnswer(invocation -> {
        User saved = invocation.getArgument(0);
        saved.setId(userId);
        return saved;
    });
    when(jwtTokenProvider.generateToken(userId)).thenReturn(expectedToken);

    // Act
    AuthResponse result = authService.register(regDTO);

    // Assert
    assertThat(result).isNotNull();
    assertThat(result.getToken()).isEqualTo(expectedToken);
    assertThat(result.getProfile()).isNotNull();
    assertThat(result.getProfile().getEmail()).isEqualTo(testEmail);
    assertThat(result.getProfile().getDisplayName()).isEqualTo(testDisplayName);

    verify(userRepository).existsByEmail(testEmail);
```

```
    verify(passwordEncoder).encode(testPassword);
    verify(userRepository).save(any(User.class));
    verify(userSettingsRepository).save(any(UserSettings.class));
    verify(jwtTokenProvider).generateToken(userId);
}
```

6) ItemServiceTest.java — создание вещи (успешный сценарий)

```
@Test
@DisplayName("createItem – успешное создание вещи")
void createItem_success() {
    // Arrange
    when(userRepository.findById(userId)).thenReturn(Optional.of(testUser));
    when(itemRepository.save(any(Item.class))).thenAnswer(invocation -> {
        Item saved = invocation.getArgument(0);
        saved.setId(itemId);
        return saved;
    });

    // Act
    ItemDTO result = itemService.createItem(userId, testItemDTO);

    // Assert
    assertThat(result).isNotNull();
    assertThat(result.getName()).isEqualTo("T-Shirt");
    assertThat(result.getCategory()).isEqualTo(Category.TOP);
    assertThat(result.getSeason()).isEqualTo(Season.SUMMER);

    verify(userRepository).findById(userId);
    verify(itemRepository).save(any(Item.class));
}
```

```
[INFO] Results:
[INFO]
[INFO] Tests run: 69, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jacoco:0.8.11:report (report) @ my-shelf-server ---
[INFO] Loading execution data file D:\УчёбаСКФУ\ПРОЕКТЫ\Курсовые\3 курс\myShelf\my-shelf-server\
[INFO] Analyzed bundle 'my-shelf-server' with 39 classes
[WARNING] Classes in bundle 'my-shelf-server' do not match with execution data. For report gener
[WARNING] Execution data for class com/myshelf/wardrobe/service/AuthService does not match.
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  35.754 s
[INFO] Finished at: 2026-11-11T20:18:02+03:00
[INFO] ------------------------------------------------------------------------
```

Рисунок 3. Результаты тестирования (mvn test) в консоли IDE

my-shelf-server

my-shelf-server

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.myshelf.wardrobe.entity		62 %		50 %	66	142	28	77	65	141	1	13
com.myshelf.wardrobe.security		0 %		0 %	13	13	38	38	8	8	2	2
com.myshelf.wardrobe.dto		80 %		n/a	15	62	30	124	15	62	0	7
com.myshelf.wardrobe.config		0 %		n/a	7	7	13	13	7	7	1	1
com.myshelf.wardrobe.controller		70 %		n/a	5	17	13	43	5	17	1	3
com.myshelf.wardrobe.service		96 %		93 %	3	47	8	188	1	32	0	3
com.myshelf.wardrobe		37 %		n/a	1	2	2	3	1	2	0	1
Total	650 of 2 207	70 %	13 of 42	69 %	110	290	132	486	102	269	5	30

Рисунок 4. Отчёт JaCoCo о покрытии тестами

Часть 2. Ручное тестирование мобильного приложения

Проверим функционал приложения по чек-листу.

1) Авторизация:

- вход/регистрация (валидные/невалидные данные);

- гостевой режим («Try without registration»).

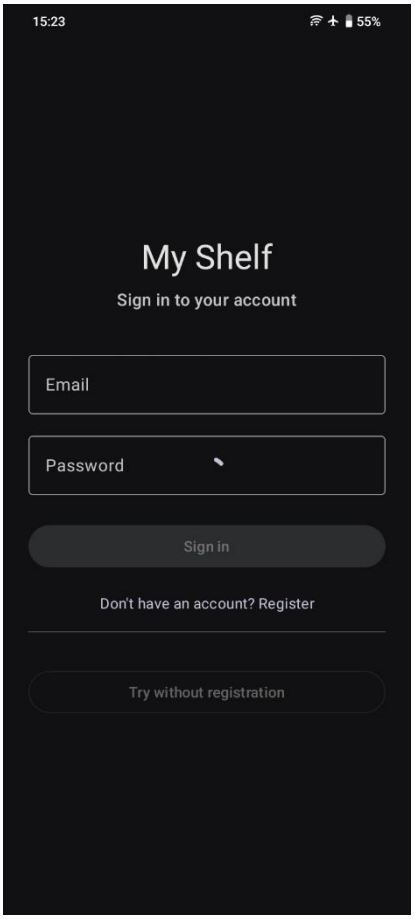


Рисунок 5. Экран авторизации

При вводе валидных данных в поля Email и Password пользователя переносит на экран «Мои образы». Если почта или пароль записаны неверно, пользователь получит предупреждение. При отсутствии данных пользователя на сервере пользователю будет предложено зарегистрироваться.

2) «Мои вещи»:

- добавление вещи: обязательные поля, выбор сезона/категории, загрузка фото;

- список: работа поиска, фильтров по категориям, скролл;

- редактирование и удаление.

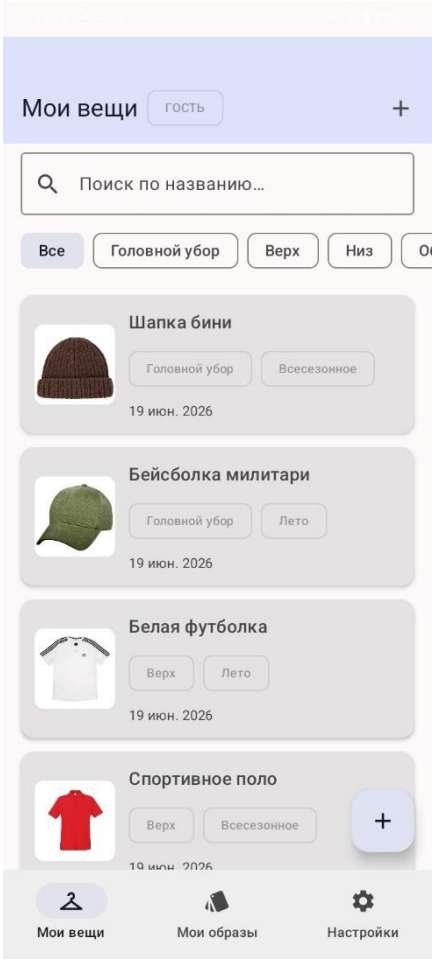


Рисунок 6. Экран "Мои вещи"

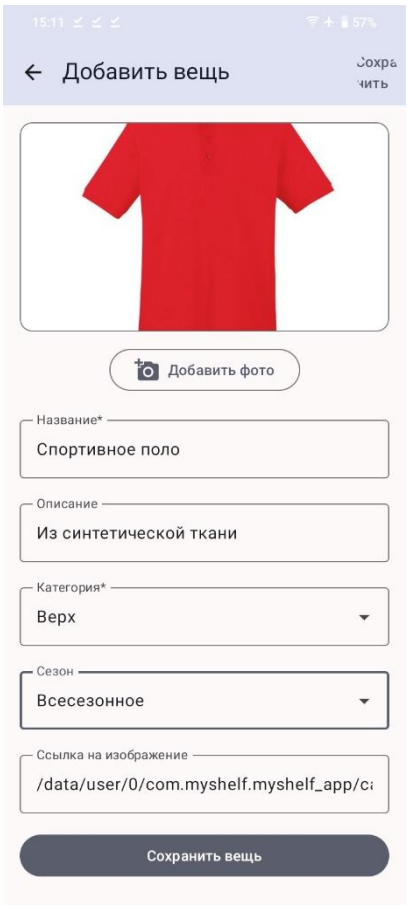


Рисунок 7. Экран создания вещи

3) «Мои образы»:

- создание: ввод данных, выбор сезона, добавление/удаление вещей;

- список: отображение карточек, работа кнопок «редактировать» и «удалить».

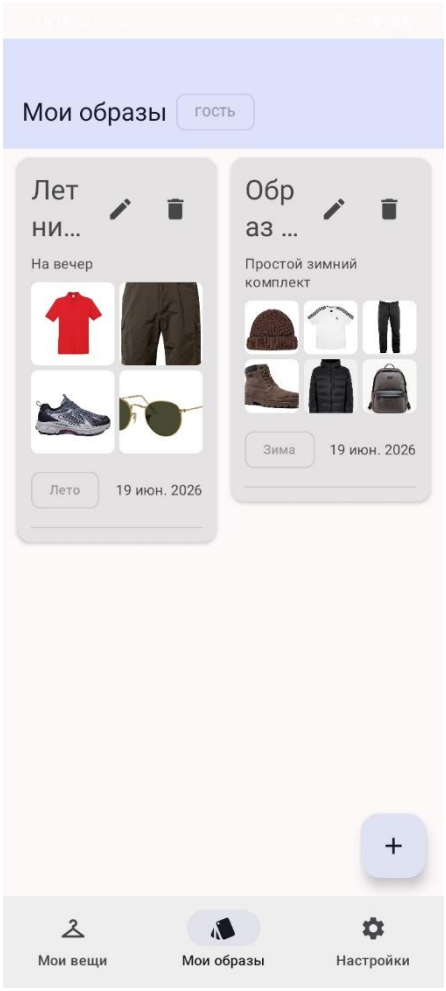


Рисунок 8. Экран "Мои образы"

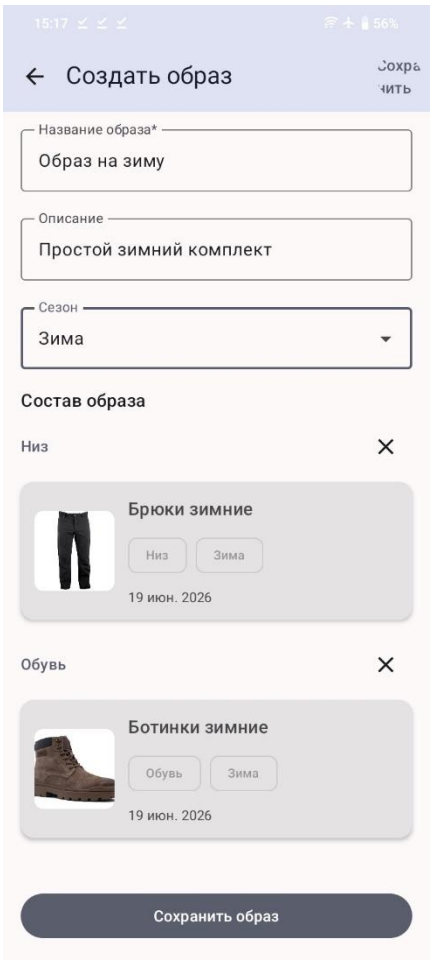


Рисунок 9. Экран создания образа

По результатам ручного тестирования приложения можно сделать вывод, что все ключевые пользовательские сценарии, включая авторизацию, ведение гардероба и создание образов, работают корректно и стабильно. Логика связности данных не нарушена (вещи корректно добавляются в образы), поиск и фильтры функционируют точно, а интерфейс остается интуитивно понятным и плавным в навигации. Критических блокирующих дефектов не выявлено, а обнаруженные минорные замечания не влияют на общую работоспособность. Приложение полностью готово к релизу или передаче на следующий этап тестирования.

Вывод:

Проделав работу, продемонстрировал применение методологии тестирования мобильного приложения.